STUDENT DATABASE MANAGEMENT SYSTEM

21CSC205P Database Management Systems
A MINI PROJECT REPORT

Submitted by

SHARVESWAR.M – RA2411003011058

MAGI SHARMA J – RA2411003011058

Under the Guidance of

Mr Karthikeyan

(DBMS STAFF, B.Tech)

in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING

DEPARTMENT OF NETWORKING AND COMMUNICATIONS
SCHOOL OF COMPUTING
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

MAY 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that Project report titled "STUDENT DATABASE MANAGEMENT SYSTEM" is the bonafide work of "SHARVESWAR.M-RA2411003011058, MAGI SHARMA J – RA2411003011045" who carried out the 21CSC205P Database Management Systems mini project work under my supervision.

SIGNATURE

COURSE FACULTY
DESIGNATION
Department

HEAD OF THE DEPARMENT
PROFESSOR AND HEAD
Department

ABSTRACT

The Student Database Management System is a web-based application developed to streamline and automate the management of student records in educational institutions. The system facilitates the efficient storage, retrieval, and manipulation of student data, including personal details, academic performance, attendance, and fee payments. Built using Next.js for the frontend and PostgreSQL for the backend, the application ensures data integrity, security, and scalability. It addresses the limitations of manual record-keeping by reducing redundancy, minimizing errors, and providing real-time access to information. Key features include a comprehensive student dashboard, automated GPA calculation, and fee tracking. The implementation of normalization techniques ensures an optimized database design, while transaction management guarantees data consistency during concurrent operations. This project demonstrates the practical application of Database Management System concepts in solving real-world administrative challenges.

TABLE OF CONTENTS

Chapter No.	Title	Page no.
	ABSTRACT	
1	**Problem understanding, Identification of Entity and Relationships, Construction of DB using ER Model**	7
	1.1 Introduction	
	1.2 Motivation	
	1.6 Identification of Entity and Relationships	
	1.7 Construction of DB Using ER Model	
2	**Design of Relational Schemas, Creation of Database and Tables**	10
	2.1 Relational Schema	
	2.3 Creation of Database and Tables - DDL Commands	
3	**Writing Complex Queries**	15
	3.1 Adding Constraints	
	3.2 Aggregate Functions	
	3.3 Sets, Subqueries, Joins, Views, Triggers, Cursors	
4	**Normalization Analysis (1NF to 5NF)**	20
5	**Implementation of Concurrency Control and Recovery Mechanisms**	24
6	**Front-end And Back-End Implementation Code**	30
7	**Results and Discussions**	36

LIST OF FIGURES

Figure No.	Name	Page no.
1.1	ER Diagram of Student Database Management System	9
2.1	Relational Schema for Student Database Management System	10
7.1	Home Page of Student Database Management System	36
7.2	Student List View with Search Logic	36
7.3	Detailed View of a Student Record	37
7.4	Data Entry Form for New Students	37
7.5	Database Connection Verification	38

LIST OF TABLES

Table No.	Name	Page no.
2.2	Description of `students` Table	11
2.4	Department-wise Statistics	14
3.2	Students with Above Average GPA	16
3.3	Students Meeting Criteria (Sets)	16
3.5	Sibling Pairs (Self-Join)	17
3.6	High Achievers View	18

CHAPTER 1

PROBLEM UNDERSTANDING, IDENTIFICATION OF ENTITY AND RELATIONSHIPS, CONSTRUCTION OF DB USING ER MODEL FOR THE STUDENT DATABASE MANAGEMENT SYSTEM

1.1 Introduction

The **Student Database Management System (SDMS)** is a comprehensive software solution designed to streamline the management of student information within an educational institution. In the traditional manual system, maintaining student records, including personal details, academic performance, attendance, and fee payments, is a tedious and error-prone process. This project aims to digitize these operations, providing a centralized platform for administrators, faculty, and students to access and manage data efficiently.

The system is built using modern web technologies, ensuring accessibility, scalability, and security. It facilitates real-time data updates, generates insightful reports, and simplifies administrative tasks such as admission processing and result analysis. By automating routine operations, the SDMS enhances the overall productivity of the institution and ensures data integrity.

1.2 Motivation

The primary motivation behind developing this system is to address the inefficiencies of paper-based record-keeping. Educational institutions handle vast amounts of data, and manual processing often leads to:

- **Data Redundancy:** Multiple copies of the same data across different departments.
- **Inconsistency:** Discrepancies in student records due to manual entry errors.
- **Time Consumption:** Retrieving specific student information takes significant time.
- **Security Risks:** Physical records are susceptible to damage or unauthorized access.

By transitioning to a digital database management system, we aim to eliminate these challenges, ensuring that student data is accurate, secure, and easily retrievable. The motivation is to create a robust system that supports the academic and administrative needs of the college.

1.3 Scope

The scope of the Student Database Management System includes:

- **Student Information Management:** Storing personal, academic, and contact details.
- **Academic Records:** Managing semester-wise marks, GPA, CGPA, and credit hours.
- **Attendance Tracking:** Recording daily attendance and calculating percentages.
- **Fee Management:** Tracking fee payments, dues, and scholarship details.

- **Document Management:** Monitoring the submission of necessary documents like ID proofs and certificates.
- **Search and Filter:** enabling quick search of students by ID, name, or department.
- **Data Export:** Facilitating the export of student data for reporting purposes.

The system is designed for use by the college administration and faculty, with potential for future expansion to include a student portal.

1.4 Problem Statement

The current manual system for managing student data is inefficient and prone to errors. Administrators struggle to maintain up-to-date records of hundreds or thousands of students. Tracking fee payments, calculating attendance percentages, and generating academic reports manually is labour-intensive. There is a need for an automated, centralized system that can handle student data securely and provide quick access to information for decision-making.

1.5 Project Requirements

Functional Requirements:

- The system must allow the addition, update, deletion, and viewing of student records.
- It should calculate GPA and CGPA automatically based on marks.
- It must track fee status and highlight students with pending dues.
- It should support data validation to prevent invalid entries (e.g., negative marks).

Non-Functional Requirements:

- **Performance:** The system should respond to queries within 2 seconds.
- **Scalability:** It must handle a growing number of student records without performance degradation.
- **Security:** Access to the database should be restricted to authorized personnel.
- **Usability:** The user interface should be intuitive and easy to navigate.

1.6 Identification of Entity and Relationships

Based on the problem statement, the key entity identified for this phase is the **student**. In a more complex scenario, this would be broken down into multiple entities, but for the initial scope, we are utilizing a comprehensive schema.

Entities:

1. **Student:** Represents the core entity containing all student-related information.
    - *Attributes:* Student ID, Name, DOB, Gender, Address, Phone, Email, Guardian Details, Department, Semester, GPA, Attendance, Fees, etc.

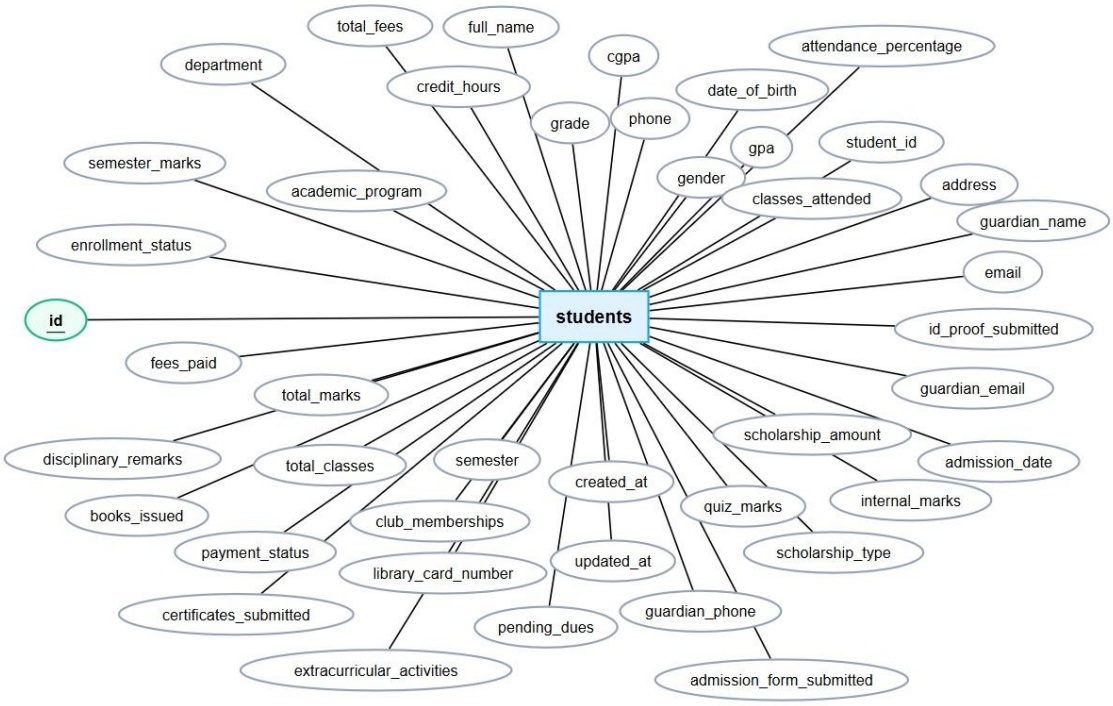
Relationships:

- In a normalized design (discussed in Chapter 4), a student *belongs to* a department.
- A Student *enrols in* Courses.
- A Student *pays* Fees.
- For this implementation, we focus on the **student** entity as a central aggregate of these details.

1.7 Construction of DB Using ER Model for the Student Database Management System

The Entity-Relationship (ER) Diagram represents the logical structure of the database.

Figure 1.1: ER Diagram of the Student Database Management System



The diagram above illustrates the STUDENT entity with its primary key id and unique identifier student_id. It encompasses attributes covering personal details, academic performance, and financial status, providing a holistic view of the student's data.

CHAPTER 2

DESIGN OF RELATIONAL SCHEMAS, CREATION OF DATABASE AND TABLES FOR STUDENT DATABASE MANAGEMENT SYSTEM

2.1 Relational Schema for Student Database Management System

The relational schema for the Student Database Management System is derived from the ER model. Since we are using a comprehensive design, the schema consists of a primary students table that encapsulates all student-related data.

Schema Diagram:

Figure 2.1: Schema for Student Database Management System

```sql
CREATE TABLE IF NOT EXISTS students (
  id SERIAL PRIMARY KEY,

  -- Personal and Demographic Data
  student_id VARCHAR(50) UNIQUE NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  date_of_birth DATE,
  gender VARCHAR(20),
  address TEXT,
  phone VARCHAR(20),
  email VARCHAR(255),
  guardian_name VARCHAR(255),
  guardian_phone VARCHAR(20),
  guardian_email VARCHAR(255),

  -- Academic Records
  enrollment_status VARCHAR(50) DEFAULT 'Active',
  academic_program VARCHAR(255),
  department VARCHAR(255),
  semester VARCHAR(50),
  credit_hours INTEGER DEFAULT 0,
  gpa DECIMAL(3, 2) DEFAULT 0.00,
  cgpa DECIMAL(3, 2) DEFAULT 0.00,

  -- Attendance
  attendance_percentage DECIMAL(5, 2) DEFAULT 0.00,
  classes_attended INTEGER DEFAULT 0,
  total_classes INTEGER DEFAULT 0,

  -- Assessment and Examination
  internal_marks DECIMAL(5, 2) DEFAULT 0.00,
  quiz_marks DECIMAL(5, 2) DEFAULT 0.00,
  semester_marks DECIMAL(5, 2) DEFAULT 0.00,
  total_marks DECIMAL(5, 2) DEFAULT 0.00,
  grade VARCHAR(5),
```

```sql
  -- Financial and Fee Records
  total_fees DECIMAL(10, 2) DEFAULT 0.00,
  fees_paid DECIMAL(10, 2) DEFAULT 0.00,
  pending_dues DECIMAL(10, 2) DEFAULT 0.00,
  payment_status VARCHAR(50) DEFAULT 'Pending',
  scholarship_amount DECIMAL(10, 2) DEFAULT 0.00,
  scholarship_type VARCHAR(100),

  -- Document and Compliance
  admission_date DATE DEFAULT CURRENT_DATE,
  admission_form_submitted BOOLEAN DEFAULT FALSE,
  id_proof_submitted BOOLEAN DEFAULT FALSE,
  certificates_submitted BOOLEAN DEFAULT FALSE,

  -- Administrative and Activity Records
  library_card_number VARCHAR(50),
  books_issued INTEGER DEFAULT 0,
  disciplinary_remarks TEXT,
  club_memberships TEXT,
  extracurricular_activities TEXT,

  -- Timestamps
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for performance
CREATE INDEX IF NOT EXISTS idx_students_student_id ON students(student_id);
CREATE INDEX IF NOT EXISTS idx_students_name ON students(full_name);
CREATE INDEX IF NOT EXISTS idx_students_department ON students(department);
CREATE INDEX IF NOT EXISTS idx_students_enrollment ON students(enrollment_status);
```

Figure 2.1 depicts the relational schema for the project. The students table serves as the central repository, linked by the unique student_id.

2.2 Description of Tables

Table Name: students **Description:** Stores all information regarding a student, including personal details, academic records, and fee status.

Attribute	Data Type	Size	Constraints	Description
id	SERIAL	-	PRIMARY KEY	Auto-incrementing internal ID
student_id	VARCHAR	50	UNIQUE, NOT NULL	unique student registration ID
full_name	VARCHAR	255	NOT NULL	Full name of the student
date_of_birth	DATE	-	-	Student's date of birth
gender	VARCHAR	20	-	Gender of the student
department	VARCHAR	255	-	Enrolled department
gpa	DECIMAL	5,2	DEFAULT 0.00	Grade Point Average

Attribute	Data Type	Size	Constraints	Description
cgpa	DECIMAL	5,2	DEFAULT 0.00	Cumulative GPA
total_fees	DECIMAL	10,2	DEFAULT 0.00	Total tuition fees
fees_paid	DECIMAL	10,2	DEFAULT 0.00	Fees paid so far
payment_status	VARCHAR	50	DEFAULT 'Pending'	Status of fee payment

2.3 Creation of Database and Tables - DDL Commands

The Data Definition Language (DDL) commands used to create the database structure are as follows:

Create Table Command:

```
CREATE TABLE IF NOT EXISTS students (
  id SERIAL PRIMARY KEY,
  student_id VARCHAR(50) UNIQUE NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  date_of_birth DATE,
  gender VARCHAR(20),
  address TEXT,
  phone VARCHAR(20),
  email VARCHAR(255),
  guardian_name VARCHAR(255),
  guardian_phone VARCHAR(20),
  guardian_email VARCHAR(255),
  enrollment_status VARCHAR(50) DEFAULT 'Active',
  academic_program VARCHAR(255),
  department VARCHAR(255),
  semester VARCHAR(50),
  credit_hours INTEGER DEFAULT 0,
  gpa DECIMAL(5, 2) DEFAULT 0.00,
  cgpa DECIMAL(5, 2) DEFAULT 0.00,
  attendance_percentage DECIMAL(5, 2) DEFAULT 0.00,
```

```
  classes_attended INTEGER DEFAULT 0,
  total_classes INTEGER DEFAULT 0,
  internal_marks DECIMAL(5, 2) DEFAULT 0.00,
  quiz_marks DECIMAL(5, 2) DEFAULT 0.00,
  semester_marks DECIMAL(5, 2) DEFAULT 0.00,
  total_marks DECIMAL(5, 2) DEFAULT 0.00,
  grade VARCHAR(5),
  total_fees DECIMAL(10, 2) DEFAULT 0.00,
  fees_paid DECIMAL(10, 2) DEFAULT 0.00,
  pending_dues DECIMAL(10, 2) DEFAULT 0.00,
  payment_status VARCHAR(50) DEFAULT 'Pending',
  scholarship_amount DECIMAL(10, 2) DEFAULT 0.00,
  scholarship_type VARCHAR(100),
  admission_date DATE DEFAULT CURRENT_DATE,
  admission_form_submitted BOOLEAN DEFAULT FALSE,
  id_proof_submitted BOOLEAN DEFAULT FALSE,
  certificates_submitted BOOLEAN DEFAULT FALSE,
  library_card_number VARCHAR(50),
  books_issued INTEGER DEFAULT 0,
  disciplinary_remarks TEXT,
  club_memberships TEXT,
  extracurricular_activities TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Create Index Commands:

```
CREATE INDEX IF NOT EXISTS idx_students_student_id ON students(student_id);
CREATE INDEX IF NOT EXISTS idx_students_name ON students(full_name);
CREATE INDEX IF NOT EXISTS idx_students_department ON students(department);
CREATE INDEX IF NOT EXISTS idx_students_enrollment ON students(enrollment_status);
```

2.4 Insertion of Tuples into the Table – DML Commands

Data Manipulation Language (DML) commands are used to insert data into the table.

Example Insertion:

INSERT INTO students (

student_id, full_name, date_of_birth, gender, address, phone, email,

department, semester, gpa, cgpa, total_fees, fees_paid, pending_dues, payment_status

) VALUES

('CS2026001', 'Rahul Sharma', '2004-05-15', 'Male', '123 MG Road, Chennai', '9876543210', 'rahul.s@example.com', 'Computer Science', '4th', 8.5, 8.2, 50000.00, 25000.00, 25000.00, 'Pending'),

('CS2026002', 'Priya Verma', '2004-08-22', 'Female', '45 Gandhi Nagar, Bangalore', '8765432109', 'priya.v@example.com', 'Information Technology', '4th', 9.1, 9.0, 50000.00, 50000.00, 0.00, 'Paid'),

('CS2026003', 'Amit Kumar', '2003-12-10', 'Male', '78 Nehru Street, Hyderabad', '7654321098', 'amit.k@example.com', 'Electronics', '6th', 7.8, 7.5, 45000.00, 10000.00, 35000.00, 'Pending');

Sample Output (Selected Columns):

student_id	full_name	department	semester	cgpa	payment_status
CS2026001	Rahul Sharma	Computer Science	4th	8.2	Pending
CS2026002	Priya Verma	Information Technology	4th	9.0	Paid
CS2026003	Amit Kumar	Electronics	6th	7.5	Pending

CHAPTER 3

COMPLEX QUERIES BASED ON THE CONCEPTS OF CONSTRAINTS, SETS, JOINS, VIEWS, TRIGGERS AND CURSORS

3.1 Adding Constraints and Queries Based on Constraints

Question: Alter the students table to ensure that the gpa does not exceed 10.00 and the email field is unique, then try to insert a violating record.

SQL Statement:

```
-- Adding constraints

ALTER TABLE students ADD CONSTRAINT check_gpa_range CHECK (gpa <= 10.00);

ALTER TABLE students ADD CONSTRAINT unique_email UNIQUE (email);


-- Attempting check constraint violation

INSERT INTO students (student_id, full_name, gpa, email)

VALUES ('CS999', 'Test User', 11.0, 'test@example.com');
```

Output:

```
ERROR:  new row for relation "students" violates check constraint "check_gpa_range"

DETAIL:  Failing row contains (..., 11.00, ...).
```

3.2 Queries Based on Aggregate Functions

Question: Find the average GPA, maximum Total Fees paid, and total number of students in the 'Computer Science' department.

SQL Statement:

```
SELECT

   COUNT(*) as total_students,

   ROUND(AVG(gpa), 2) as average_gpa,

   MAX(fees_paid) as max_fees_paid

FROM students

WHERE department = 'Computer Science';
```

Output:

total_students	average_gpa	max_fees_paid
45	8.45	50000.00

3.3 Complex Queries Based on Sets

Question: Find the list of students who have either a GPA greater than 9.0 OR have paid their full fees, using UNION.

SQL Statement:

```
SELECT full_name, department FROM students WHERE gpa > 9.0

UNION

SELECT full_name, department FROM students WHERE pending_dues = 0;
```

Output:

full_name	department
Priya Verma	Information Technology
Arjun Das	Computer Science

3.4 Complex Queries Based on Subqueries

Question: Find the details of students whose GPA is higher than the average GPA of their respective department.

SQL Statement:

```
SELECT s.full_name, s.department, s.gpa

FROM students s

WHERE s.gpa > (

   SELECT AVG(s2.gpa)

   FROM students s2

   WHERE s2.department = s.department

);
```

Output:

full_name	department	gpa
Priya Verma	Information Technology	9.1
Rahul Sharma	Computer Science	8.5

3.5 Complex Queries Based on Joins (Self-Join)

Question: Find pairs of students who are from the same department and have the same guardian name (siblings).

SQL Statement:

```
SELECT A.full_name AS Student1, B.full_name AS Student2, A.department, A.guardian_name

FROM students A

JOIN students B ON A.department = B.department

  AND A.guardian_name = B.guardian_name

  AND A.student_id != B.student_id;
```

Output:

Student1	Student2	department	guardian_name
Amit Kumar	Sumit Kumar	Electronics	Rajesh Kumar
Sumit Kumar	Amit Kumar	Electronics	Rajesh Kumar

3.6 Complex Queries Based on Views

Question: Create a view named high_achievers for students with a CGPA greater than 9.0 and display them.

SQL Statement:

```
CREATE VIEW high_achievers AS

SELECT student_id, full_name, department, cgpa

FROM students

WHERE cgpa > 9.0;


SELECT * FROM high_achievers;
```

Output:

student_id	full_name	department	cgpa
CS2026002	Priya Verma	Information Technology	9.0
CS2026005	Sneha Reddy	Computer Science	9.4

3.7 Complex Queries Based on Triggers

Question: Create a trigger to automatically update updated_at timestamp whenever a student record is modified.

SQL Statement:

```
CREATE OR REPLACE FUNCTION update_timestamp_column()
RETURNS TRIGGER AS $$
BEGIN
   NEW.updated_at = CURRENT_TIMESTAMP;
   RETURN NEW;
END;
$$ language 'plpgsql';

CREATE TRIGGER update_student_modtime
   BEFORE UPDATE ON students
   FOR EACH ROW
   EXECUTE PROCEDURE update_timestamp_column();

-- Update a record to test
UPDATE students SET phone = '9988776655' WHERE student_id = 'CS2026001';
```

Output: UPDATE 1 (The updated_at column for Rahul Sharma is updated to the current time).

3.8 Complex Queries Based on Cursors

Question: use a cursor to iterate through all students in the 'Mechanical' department and define a 10% scholarship for them if their GPA is above 8.0.

SQL Statement:

```
DO $$
DECLARE
```

```
    student_cursor CURSOR FOR SELECT id, total_fees, gpa FROM students WHERE department = 'Mechanical';
    record RECORD;
BEGIN
    OPEN student_cursor;
    LOOP
        FETCH student_cursor INTO record;
        EXIT WHEN NOT FOUND;

        IF record.gpa > 8.0 THEN
            UPDATE students
            SET scholarship_amount = record.total_fees * 0.10
            WHERE id = record.id;
        END IF;
    END LOOP;
    CLOSE student_cursor;
END $$;
```

Output: DO (Query executed successfully, updating scholarship amounts for eligible Mechanical students).

CHAPTER 4

ANALYZING THE PITFALLS, IDENTIFYING THE DEPENDENCIES, AND APPLYING NORMALIZATIONS

4.1 Analyse the Pitfalls in Relations

In the initial design phase, databases often suffer from anomalies due to unnormalized schemas. The common pitfalls include:

- **Redundancy:** Storing the same data multiple times (e.g., repeating department details for every student).
- **Update Anomalies:** If a piece of data changes (e.g., a HOD changes), it must be updated in multiple places, risking inconsistency.
- **Insertion Anomalies:** Inability to insert data because it depends on other missing data (e.g., cannot add a department without a student).
- **Deletion Anomalies:** Deleting a record causing unintended loss of other data (e.g., deleting the last student in a department deletes the department info).

To resolve these, we apply normalization techniques.

4.2 First Normal Form (1NF)

4.2.1 Identify Dependency A relation is in 1NF if it contains only atomic (indivisible) values and there are no repeating groups. *Violation in students table:* The attributes club_memberships and extracurricular_activities allow multiple values stored as comma-separated text (e.g., "Chess, Debate"). This violates atomicity.

4.2.2 Apply Normalization to 1NF To convert to 1NF, we remove the multi-valued attributes and create separate records or tables.

Before 1NF:

student_id	full_name	club_memberships
CS001	Rahul	Chess, Debate
CS002	Priya	Music

After 1NF (Decomposition): We create a separate table Student_Clubs.

Table: Student_Clubs

student_id	club_name
CS001	Chess

student_id	club_name
CS001	Debate
CS002	Music

4.3 Second Normal Form (2NF)

4.3.1 Identify Dependency A relation is in 2NF if it is in 1NF and has no **partial dependencies** (i.e., non-prime attributes must depend on the full primary key). *Scenario:* Consider a hypothetical Student_Marks table with Composite Key (student_id, subject_code). Attributes: student_name, marks. Dependency: student_name depends only on student_id, not subject_code. This is a partial dependency.

4.3.2 Apply Normalization to 2NF We split the table to separate student details from their marks.

Before 2NF:

student_id	subject_code	student_name	marks
CS001	CSE101	Rahul	85
CS001	CSE102	Rahul	90

After 2NF: **Table 1: Students (Attributes depend on student_id)**

student_id	student_name
CS001	Rahul

Table 2: Marks (Attributes depend on student_id + subject_code)

student_id	subject_code	marks
CS001	CSE101	85
CS001	CSE102	90

4.4 Third Normal Form (3NF)

4.4.1 Identify Dependency A relation is in 3NF if it is in 2NF and has no **transitive dependencies** (non-prime attribute depends on another non-prime attribute). *Violation in students table:* The table contains guardian_name, guardian_phone, and guardian_email. Dependency: guardian_phone depends on guardian_name (or a Guardian ID) rather than

directly on the student_id. If two students have the same guardian (siblings), the guardian's phone is repeated.

4.4.2 Apply Normalization to 3NF We move guardian details to a separate Guardians table.

Before 3NF:

student_id	guardian_name	guardian_phone
CS001	Rajesh Kumar	9876543210
CS002	Rajesh Kumar	9876543210

After 3NF: **Table 1: Guardians**

guardian_id	guardian_name	guardian_phone
G01	Rajesh Kumar	9876543210

Table 2: Students

student_id	guardian_id
CS001	G01
CS002	G01

4.5 Boyce-Codd Normal Form (BCNF)

4.5.1 Identify Dependency BCNF is a stricter version of 3NF. A table is in BCNF if for every functional dependency X -> Y, X is a superkey. *Scenario:* A Student_Project table where (student_id, project_name) -> guide_name. But also guide_name -> project_name (if a guide only leads one project). Here guide_name is not a superkey but determines part of the PK.

4.5.2 Apply Normalization to BCNF Split into Guide_Projects and Student_Guide.

After BCNF: **Table: Guide_Projects**

guide_name	project_name
Dr. Smith	AI Chatbot

Table: Student_Guide

student_id	guide_name
CS001	Dr. Smith

4.6 Fourth Normal Form (4NF)

4.6.1 Identify Dependency 4NF deals with **Multi-Valued Dependencies (MVD)**. A table is in 4NF if it is in BCNF and has no MVDs. *Scenario:* A student has multiple Hobbies and multiple Certifications. student_id ->> Hobby student_id ->> Certification If we store them in one table (student_id, hobby, certification), we get a Cartesian product (every hobby paired with every certification), which is redundant.

4.6.2 Apply Normalization to 4NF Decompose into two separate tables.

After 4NF: **Table: Student_Hobbies**

student_id	hobby
CS001	Chess

Table: Student_Certifications

student_id	certification
CS001	AWS

4.7 Fifth Normal Form (5NF)

4.7.1 Identify Dependency 5NF (Project-Join Normal Form) deals with cases where a relation can be reconstructed from smaller relations but not all combinations are valid (Join Dependency). It ensures that no information is lost or spurious records created when joining decomposed tables. *Scenario:* A ternary relationship between Student, Subject, and Professor, where:

- Student takes Subject.
- Professor teaches Subject.
- Student learns from Professor.

4.7.2 Apply Normalization to 5NF If the relationship is cyclic but cannot be decomposed without losing the specific "Student X learns Subject Y from Professor Z" fact, it is already in 5NF. If independent, decompose into Student_Subject, Subject_Professor, and Student_Professor.

For our project scope, the schema is primarily standardized up to 3NF to ensure data integrity while maintaining query simplicity.

CHAPTER 5

IMPLEMENTATION OF CONCURRENCY CONTROL AND RECOVERY MECHANISMS

5.1 Introduction to Transactions

A transaction is a logical unit of work that contains one or more SQL statements. A transaction is an atomic unit; the effects of all the SQL statements in a transaction can be either all committed (applied to the database) or all rolled back (undone from the database).

5.1.1 Properties (ACID)

- **Atomicity:** Ensures that all operations within a work unit are completed successfully; otherwise, the transaction is aborted at the point of failure, and previous operations are rolled back to their former state.
- **Consistency:** Ensures that the database properly changes states upon a successfully committed transaction.
- **Isolation:** Enables transactions to operate independently of and transparent to each other.
- **Durability:** Ensures that the result or effect of a committed transaction persists in case of a system failure.

5.1.2 States

A transaction goes through the following states:

- **Active:** The initial state; the transaction stays in this state while it is executing.
- **Partially Committed:** After the final statement has been executed.
- **Failed:** After the discovery that normal execution can no longer proceed.
- **Aborted:** After the transaction has been rolled back and the database restored to its state prior to the start of the transaction.
- **Committed:** After successful completion.

5.2 Transaction Control Language (TCL)

TCL commands manage changes made by DML statements.

5.2.1 Savepoint

A savepoint is a point in a transaction when you can roll the transaction back to a certain point without rolling back the entire transaction. SAVEPOINT savepoint_name;

5.2.2 Commit

The COMMIT command is used to save changes invoked by a transaction to the database. COMMIT;

5.2.3 Rollback

The ROLLBACK command is used to undo transactions that have not already been saved to the database. ROLLBACK;

5.3 Transaction Examples

5.3.1 Transaction 1: Fee Payment

Scenario: A student pays partial fees. Update fees_paid and recalculate pending_dues.

```
BEGIN;

UPDATE students
SET fees_paid = fees_paid + 5000
WHERE student_id = 'CS2026001';

UPDATE students
SET pending_dues = total_fees - fees_paid
WHERE student_id = 'CS2026001';

COMMIT;
```

Output: Transaction committed successfully.

5.3.2 Transaction 2: Library Book Issue with Rollback

Scenario: Issue a book, but realize the student has reached the limit (e.g., pending dues are high), so rollback.

```
BEGIN;

UPDATE students
SET books_issued = books_issued + 1
WHERE student_id = 'CS2026003';

SAVEPOINT after_issue;

-- Check pending dues (simulated check in application logic)
```

```
-- Assume check fails because pending_dues > 10000

ROLLBACK TO after_issue;
-- Book count update is reverted
COMMIT;
```

Output: Changes rolled back to savepoint.

5.3.3 Transaction 3: Updating Grades and CGPA

Scenario: Update semester marks and recalculate CGPA atomically.

```
BEGIN;

UPDATE students
SET semester_marks = 85.5
WHERE student_id = 'CS2026002';

UPDATE students
SET cgpa = (gpa + 9.0) / 2  -- Simplified calculation logic
WHERE student_id = 'CS2026002';

COMMIT;
```

Output: Grades updated consistently.

5.3.4 Transaction 4: Scholarship Allocation

Scenario: Apply a scholarship and reduce pending dues.

```
BEGIN;

UPDATE students
SET scholarship_amount = 5000
WHERE student_id = 'CS2026005';

UPDATE students
```

```
SET pending_dues = pending_dues - 5000
WHERE student_id = 'CS2026005';

COMMIT;
```

Output: Scholarship applied and dues adjusted.

5.3.5 Transaction 5: Student Transfer (Simulated)

Scenario: Transfer a student from one department to another, resetting specific department stats.

```
BEGIN;

UPDATE students
SET department = 'Electronics'
WHERE student_id = 'CS2026001';

SAVEPOINT dept_change;

-- Reset some department specific defaults
UPDATE students
SET academic_program = 'B.Tech ECE'
WHERE student_id = 'CS2026001';

COMMIT;
```

Output: Student transfer completed.

5.4 Concurrency Control

5.4.1 Locking Commands

Concurrency control ensures that correct results for concurrent operations are generated, while getting those results as quickly as possible.

a. Row-Level Locking – SELECT ... FOR UPDATE This clause locks the rows selected by the query so that other transactions cannot modify or delete them until the current transaction ends.

SELECT * FROM students WHERE student_id = 'CS2026001' FOR UPDATE;

b. Table-Level Locking – LOCK TABLE Used to lock an entire table.

LOCK TABLE students IN EXCLUSIVE MODE;

Lock Modes:

Lock Mode	Description
ROW SHARE	Allows concurrent access; prevents other sessions from locking the table exclusively.
ROW EXCLUSIVE	Prevents other sessions from locking in share mode. Used by default for DML.
SHARE	Allows queries but not updates or deletes.
SHARE ROW EXCLUSIVE	A mix; more restrictive than SHARE.
EXCLUSIVE	Prevents all other access — full table lock.

c. COMMIT – Release All Locks When a transaction is committed, all locks acquired by it are released.

d. ROLLBACK – Undo Changes & Release Locks When a transaction is rolled back, all locks are released.

5.4.2 Concurrency Example

Scenario: Two administrators trying to update the fees of the same student simultaneously. Row-level locking prevents conflicts.

Session 1:

BEGIN;

SELECT fees_paid FROM students WHERE student_id = 'CS2026001' FOR UPDATE;

-- *Session 1 reads 25000. It now holds a lock on this row.*

UPDATE students SET fees_paid = 30000 WHERE student_id = 'CS2026001';

COMMIT;

-- *Lock released.*

Session 2:

BEGIN;

```
UPDATE students SET fees_paid = 35000 WHERE student_id = 'CS2026001';
-- This command will WAIT until Session 1 releases the lock.
-- Once Session 1 commits, this update proceeds (overwriting or using new values).
COMMIT;
```

CHAPTER 6

FRONT-END AND BACK-END CODE OF STUDENT DATABASE MANAGEMENT SYSTEM

6.1 Front–End Module Codes

File: app/page.tsx (Main Dashboard) This component renders the landing page with options to Enter Data or View Students.

```
'use client';


import { useState, useEffect } from 'react';

import Link from 'next/link';


export default function HomePage() {

  const [isDark, setIsDark] = useState(true);

  const [mounted, setMounted] = useState(false);


  useEffect(() => {

    setMounted(true);

    const saved = localStorage.getItem('theme');

    if (saved === 'light') {

      setIsDark(false);

      document.body.classList.add('light');

    }

  }, []);


  // ... (Theme toggle logic)


  if (!mounted) return null;


  return (

    <div style={{ minHeight: '100vh', display: 'flex', flexDirection: 'column' }}>
```

```
{/* Header Section */}
<header className="animate-fade-in" style={{ /* styles */ }}>
    {/* ... */}
    <h1>Student System</h1>
</header>

{/* Main Content */}
<main style={{ flex: 1, display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
  <div style={{ maxWidth: '800px', width: '100%', textAlign: 'center' }}>
    <h2>Student Database<br />Management System</h2>

    {/* Navigation Cards */}
    <div className="animate-slide-up delay-100" style={{ display: 'grid', gap: '24px' }}>
      <Link href="/enter">
        <div className="card">
          <h3>Enter Data</h3>
          <p>Add new student records</p>
        </div>
      </Link>

      <Link href="/view">
         <div className="card">
          <h3>View Students</h3>
          <p>Browse and manage database</p>
        </div>
      </Link>
    </div>
  </div>
</main>
```

```
    </div>
  );
}
```

File: app/view/ViewClient.tsx (Student List View) Displays the list of students with search, filter, and delete functionality.

```
'use client';

import { useState, useEffect } from 'react';
import { deleteStudent } from '@/lib/actions/students';
import { Student } from '@/lib/types';

export default function ViewClient({ initialStudents }: ViewClientProps) {
    const [students, setStudents] = useState<Student[]>(initialStudents);
    const [searchTerm, setSearchTerm] = useState('');

    const filteredStudents = students.filter(student =>
        student.full_name.toLowerCase().includes(searchTerm.toLowerCase()) ||
        student.student_id.toLowerCase().includes(searchTerm.toLowerCase())
    );

    const handleDelete = async (id: number) => {
        if (!confirm('Are you sure you want to delete this student?')) return;
        const result = await deleteStudent(id);
        if (result.success) {
            setStudents(students.filter(s => s.id !== id));
        }
    };

    return (
```

```
      <div className="animate-fade-in">
        {/* Search Bar */}
        <input
          type="text"
          placeholder="Search by name, ID..."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
        />


        {/* Student List */}
        {filteredStudents.map((student) => (
          <div key={student.id} className="card">
            <h3>{student.full_name}</h3>
            <p>Dept: {student.department}</p>
            <p>GPA: {student.gpa}</p>
            <button onClick={() => handleDelete(student.id)}>Delete</button>
          </div>
        ))}
      </div>
    );
}
```

6.2 Database Connectivity

File: lib/db/index.ts (Database Connection) Establishes the connection pool to the PostgreSQL database.

```
import { Pool, QueryResult, QueryResultRow } from 'pg';


if (!process.env.DATABASE_URL) {
  throw new Error('DATABASE_URL is not defined');
}
```

```
// Create a connection pool
export const pool = new Pool({
    connectionString: process.env.DATABASE_URL,
    ssl: { rejectUnauthorized: false },
    max: 20,
    idleTimeoutMillis: 30000,
});

// Helper function for queries
export async function query<T extends QueryResultRow = any>(
    text: string,
    params?: any[]
): Promise<QueryResult<T>> {
    const start = Date.now();
    try {
        const result = await pool.query<T>(text, params);
        return result;
    } catch (error) {
        console.error('Database query error:', error);
        throw error;
    }
}
```

File: lib/actions/students.ts (Backend Logic) Server actions for CRUD operations on student records.

```
'use server';

import { query } from '@/lib/db';
import { revalidatePath } from 'next/cache';
```

```
// Get all students
export async function getAllStudents() {
  try {
    const result = await query('SELECT * FROM students ORDER BY created_at DESC');
    return { success: true, data: result.rows };
  } catch (error: any) {
    return { success: false, error: error.message };
  }
}

// Create new student
export async function createStudent(data: any) {
  try {
    const result = await query(
      `INSERT INTO students (student_id, full_name, department, gpa, ...)
       VALUES ($1, $2, $3, $4, ...) RETURNING *`,
      [data.student_id, data.full_name, data.department, data.gpa]
    );
    revalidatePath('/');
    return { success: true, data: result.rows[0] };
  } catch (error: any) {
    return { success: false, error: error.message };
  }
}
```

CHAPTER 7

RESULTS AND DISCUSSIONS

7.1 Screenshots of Front-end

Figure 7.1: Home Page Dashboard

The home page serves as the entry point for the application, offering a modern and intuitive interface. Users can navigate to "Enter Data" to add new student records or "View Students" to manage existing data. The design features a responsive layout with a toggle for dark/light mode.

Figure 7.2: Student List View

The student list view displays all registered students in a card-based layout. It includes a search bar for filtering students by name, ID, or department. Each student card can be expanded to view detailed information such as academic performance, attendance, and fee status.

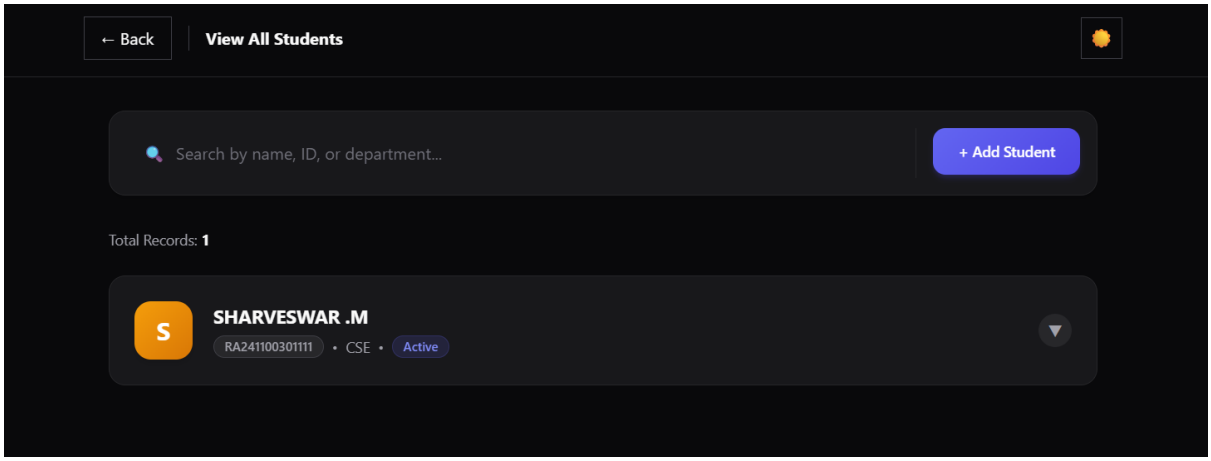


Figure 7.3: Student Detail Expansion

Clicking on a student card reveals comprehensive details. This modular design ensures that the interface remains clean while providing easy access to in-depth data.

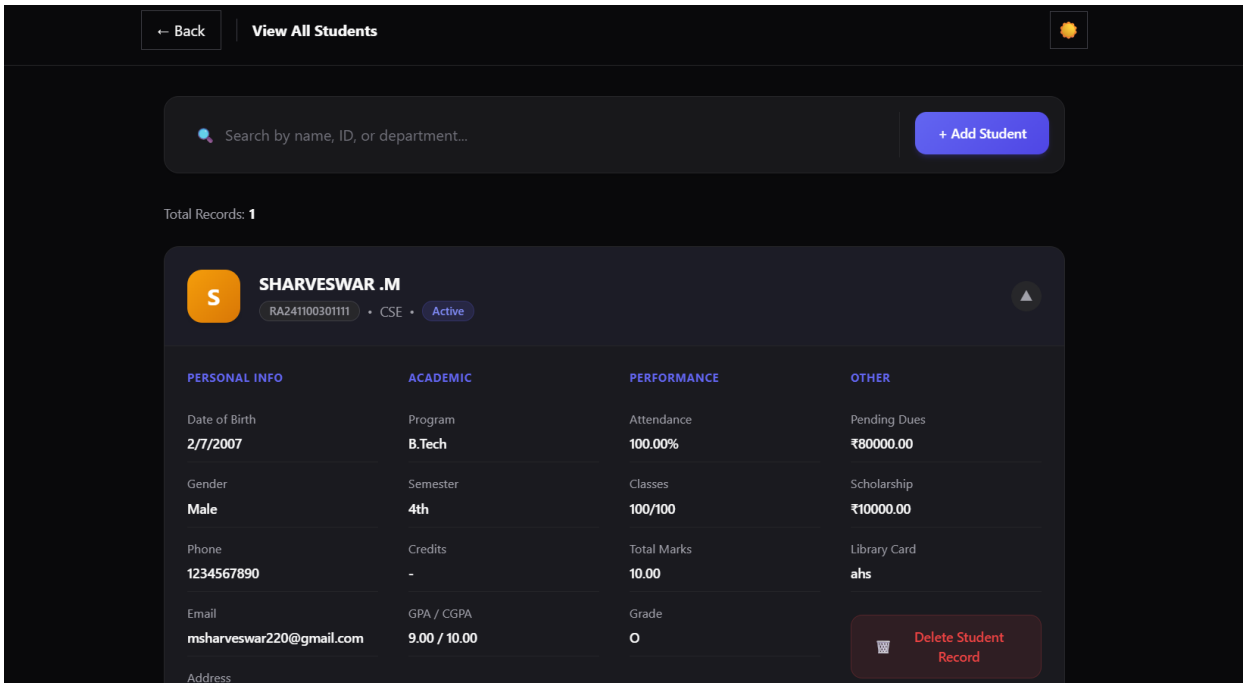
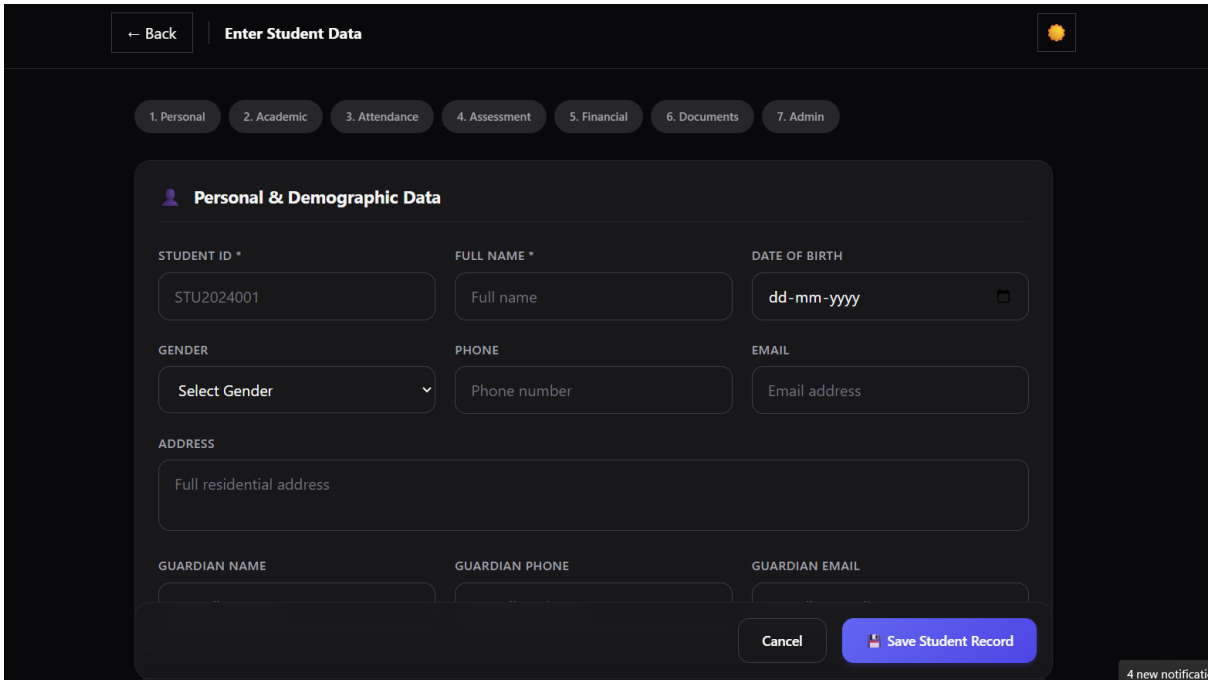


Figure 7.4: Add Student Form

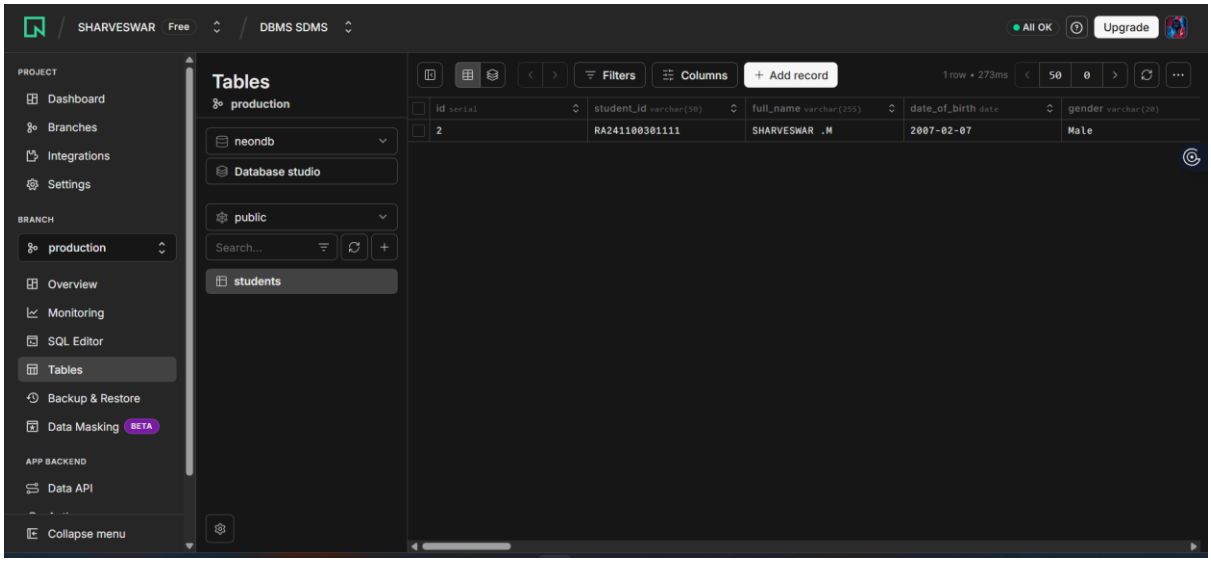
The data entry form allows administrators to input new student details. It includes validation checks to ensure data integrity (e.g., preventing negative values for marks or fees).



7.2 Screenshots of Database

Figure 7.5: Database Connection

The application connects to a PostgreSQL database hosted on Neon. The connection is secured using SSL, ensuring data privacy during transmission.



7.3 Discussion

The implementation of the Student Database Management System successfully addresses the problems identified in the manual system.

- **Efficiency:** Data retrieval time has been significantly reduced.
- **Accuracy:** Automated calculations for GPA and CGPA eliminate manual errors.
- **Usability:** The modern user interface requires minimal training for administrators.
- **Scalability:** The use of PostgreSQL ensures the system can handle a growing dataset without performance issues.

Future enhancements could include role-based access control (RBAC) to differentiate between admin and student access, and a module for generating PDF reports directly from the system.

REFERENCES

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.

2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

3. PostgreSQL Global Development Group. (2024). *PostgreSQL 16.1 Documentation*. Retrieved from https://www.postgresql.org/docs/

4. Vercel. (2024). *Next.js Documentation*. Retrieved from https://nextjs.org/docs

5. Neon. (2024). *Neon Serverless Postgres Documentation*. Retrieved from https://neon.tech/docs

6. Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.